Open University of Kenya

MinIO for Data Lakes

Presented by: Group 5

What is MinIO?

MinIO is a high-performance, Simple Storage Service (S3) - compatible object storage solution released under the GNU AGPL v3.0 license. Designed for speed and scalability, it powers AI/ML, analytics, and data-intensive workloads with industry-leading performance.

Key Concepts:

Erasure Coding - It's a data protection method that breaks data into fragments, expands them with redundant parity blocks, and stores them across different, independent locations (disks, nodes, or locations). It ensures that the original data can be fully reconstructed from a subset of these fragments, even if multiple storage devices fail simultaneously.

Quorum & Minimum Disks - A minimum number of disks is required to read/write data to ensure data consistency and integrity

Key reasons to choose an on-prem data lake over a cloud-based data lake, especially for public health, government, and regulated environments

1. Data Sovereignty & Regulatory Compliance

To comply with national data protection laws and health data regulations, where data never leaves your physical premises or should never be hosted outside the respective country.

2. Stronger Control Over Sensitive Data

To get full control over physical access, network isolation, encryption keys, and it's easier to implement air-gapped environments

3. Predictable & Lower Long-Term Cost.

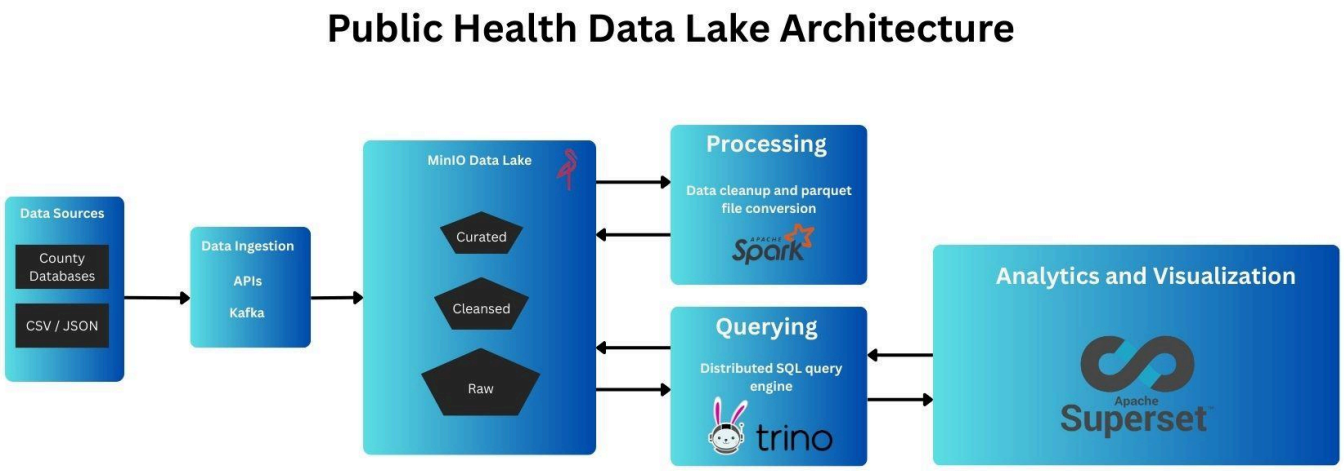
On-premises solution incurs a fixed upfront cost while on cloud costs grow fast at scale - Per-query costs, Data egress fees, API request charges, Compute engine costs, etc

4. Avoid Vendor Lock-in
Uses open standards; Parquet, SQL, S3-compatible APIs, and it's easy to migrate to another vendor or go hybrid later. Cloud-native services often trap you in proprietary formats.

5. High Performance for Local Data
When users and data are in the same location, there is no network latency to cloud regions. Hence, it's faster for repeated queries and dashboards refreshed every few seconds.

6. No Internet Dependency

Hence, it will work in remote areas, low-bandwidth regions, and unreliable connectivity environments. Hence, it's essential for rural health systems, disaster response centers, and military/emergency operations.



Installation Steps:

1. Prerequisites

   - Docker installation
   - Python 3.7 or higher

2. Make Directories - to simulate multiple disks

mkdir minio-test
cd minio-test

mkdir -p data/disk1 data/disk2 data/disk3 data/disk4 trino/catalog trino/metastore

3. Install dependencies

Create a requirements.txt file and add below dependencies

```
pandas
minio
numpy
openpyxl
pyspark
trino
sqlalchemy-trino
```

Then run command (pip install -r requirements.txt) to install dependencies

4. Setup properties files

i) Trino config (Saved as follows trino/config.properties)

```
coordinator=true
node-scheduler.include-coordinator=true
http-server.http.port=8080
query.max-memory=5GB
query.max-memory-per-node=1GB
discovery-server.enabled=true
discovery.uri=http://localhost:8080
```

ii) Hive properties (Saved as follows trino/catalog/hive.properties)

```
connector.name=hive

hive.metastore=file
hive.metastore.catalog.dir=file:///etc/trino/hive-metastore

hive.s3.endpoint=http://minio:9000
hive.s3.aws-access-key=minioadmin
hive.s3.aws-secret-key=minioadmin
hive.s3.path-style-access=true
hive.s3.ssl.enabled=false
```

5. Docker Compose setup (docker-compose.yaml)

```yaml
services:
 # ======================
 # MINIO
 # ======================

 minio:
  image: minio/minio:RELEASE.2024-01-16T16-07-38Z
  platform: linux/arm64
  container_name: minio
  command: server --console-address ":9001" http://minio/data/disk{1...4}
  hostname: minio
  ports:
   - "9000:9000"
   - "9001:9001"
  environment:
   MINIO_ROOT_USER: minioadmin
   MINIO_ROOT_PASSWORD: minioadmin
   MINIO_MIN_FREE_SPACE: 1MB # reduce threshold for testing
  volumes:
   - type: bind
     source: ./data/disk1
     target: /data/disk1
   - type: bind
     source: ./data/disk2
     target: /data/disk2
   - type: bind
     source: ./data/disk3
     target: /data/disk3
   - type: bind
     source: ./data/disk4
     target: /data/disk4

 # ======================
 # POSTGRES for Superset
 # ======================
 postgres:
  image: postgres:15-alpine
```

```yaml
  platform: linux/arm64
  environment:
    POSTGRES_USER: superset
    POSTGRES_PASSWORD: superset
    POSTGRES_DB: superset
  ports:
    - "5432:5432"
  volumes:
    - postgres-data:/var/lib/postgresql/data

# =======================
# SPARK
# =======================
spark-master:
  image: apache/spark:3.5.0
  platform: linux/arm64
  container_name: spark-master
  command: /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master
  ports:
    - "8080:8080"
    - "7077:7077"
  depends_on:
    - minio

spark-worker:
  image: apache/spark:3.5.0
  platform: linux/arm64
  container_name: spark-worker
  command: /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker spark://spark-master:7077
  depends_on:
    - spark-master

# Hive metastore
hive-metastore:
  image: apache/hive:3.1.3
  platform: linux/amd64
  container_name: hive-metastore
  environment:
    SERVICE_NAME: metastore
    DB_DRIVER: derby
  ports:
    - "9083:9083"
```

```yaml
# ======================
# TRINO
# ======================
trino:
  image: trinodb/trino:438
  platform: linux/arm64
  container_name: trino
  ports:
    - "8085:8080"
  volumes:
    - ./trino/catalog:/etc/trino/catalog
    - ./trino/metastore:/etc/trino/hive-metastore
  depends_on:
    - hive-metastore
    - minio

# ======================
# SUPERSET
# ======================
superset:
  image: apache/superset:3.1.0
  platform: linux/arm64
  container_name: superset
  ports:
    - "8088:8088"
  environment:
    SUPERSET_SECRET_KEY: supersecretkey
    DATABASE_URL: postgresql+psycopg2://superset:superset@postgres:5432/superset
  depends_on:
    - postgres
  command: >
    /bin/bash -c "
    superset db upgrade &&
    superset fab create-admin --username admin --firstname Superset --lastname Admin --email admin@local.com --password admin &&
    superset init &&
    superset run -h 0.0.0.0 -p 8088
    "

volumes:
  postgres-data:
```

Docker Commands:

```
# Start MinIO in detached mode
docker compose up -d
# Check status
docker compose ps
# View logs (follow mode)
docker compose logs -f minio1
# Stop MinIO
docker compose down
```

MinIO Client (mc) - Setup

```
# Install via Homebrew
brew install minio/stable/mc
# Verify installation
mc --version
# Configure alias - Add MinIO server to mc config
mc alias set mylocal http://localhost:9000 minioadmin minioadmin
# Verify connection
mc admin info mylocal
```

MinIO Client (mc) - Commands

```
mc ls mylocal                # List buckets
mc mb mylocal/raw            # Create bucket
mc cp file.csv mylocal/datalake/ # Upload file
```

mc cat mylocal/datalake/file.csv # View file

mc rm mylocal/datalake/file.csv # Delete file

mc version enable mylocal/datalake # Set bucket versioning (optional but recommended)

mc mb mylocal/datalake/raw/ # Create "folders" using prefixes

mc cp sales.csv mylocal/datalake/raw/sales/ # Copy file and create folders

mc cp --recursive ./data/ mylocal/datalake/raw/ # Recursive upload (entire directory)

mc cp --threads 4 large_dataset.parquet mylocal/datalake/ # Parallel uploads (faster for large files)

mc stat mylocal/datalake/customers.csv # Check object info

Access URLs:

MinIO http://localhost:9001

Spark UI http://localhost:8080

Trino http://localhost:8085

Kafka http://localhost:9092

Kafka UI http://localhost:8090

Superset http://localhost:8088

Superset Admin setup

```
docker compose exec superset \
        superset fab create-admin \
        --username admin \
        --firstname Superset \
        --lastname Admin \
        --email admin@superset.com \
        --password admin
```

ERROR: Could not load database driver: TrinoEngineSpec

```
docker exec -it superset bash -lc "pip show superset-trino sqlalchemy-trino trino || true"
```

docker exec -it superset bash -lc "pip install --no-cache-dir superset-trino sqlalchemy-trino trino"
docker restart superset

Trino setup

docker exec -it trino trino --server localhost:8080

Show catalogs;

create schema

CREATE SCHEMA IF NOT EXISTS hive.public_health
WITH (location = 's3a://curated/');

SHOW SCHEMAS FROM hive;

```
CREATE TABLE IF NOT EXISTS hive.public_health.patient_visits (
        patient_id varchar,
        hospital varchar,
        county varchar,
        visit_date date,
        disease varchar,
        diagnosis_code varchar
)
WITH (
        external_location = 's3a://curated/',
        format = 'PARQUET'
);
```

Test Trino SQL Query

SELECT * FROM hive.public_health.patient_visits LIMIT 10;

Superset Analytics Queries:

i) Disease distribution query

```
SELECT disease AS disease,
       sum(cases) AS "SUM(cases)"
FROM
  (SELECT disease,
          COUNT(*) AS cases
   FROM hive.public_health.patient_visits
   GROUP BY disease
   ORDER BY cases DESC) AS virtual_table
GROUP BY disease
ORDER BY "SUM(cases)" DESC
LIMIT 1000;
```

ii) Outbreak spike detection

```
SELECT county AS county,
       disease AS disease,
       visit_date AS visit_date,
```

```
       cases AS cases,
       cases_7d AS cases_7d
FROM
  (WITH daily AS
     (SELECT county,
             disease,
             visit_date,
             COUNT(*) AS cases
      FROM hive.public_health.patient_visits
      GROUP BY county,
               disease,
               visit_date),
        rolling AS
     (SELECT *,
             SUM(cases) OVER (PARTITION BY county,
                                           disease
                              ORDER BY visit_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS cases_7d
      FROM daily) SELECT *
   FROM rolling
   WHERE cases_7d > 1000
   ORDER BY cases_7d DESC) AS virtual_table
LIMIT 1000;
```

iii) Malaria Outbreak Bar Chart

```
SELECT county AS county,
       sum(malaria_cases) AS "SUM(malaria_cases)"
FROM
  (SELECT visit_date,
          county,
          COUNT(*) AS malaria_cases
   FROM hive.public_health.patient_visits
   WHERE lower(disease) = 'malaria'
   GROUP BY visit_date,
            county
   ORDER BY visit_date,
            county) AS virtual_table
GROUP BY county
ORDER BY "SUM(malaria_cases)" DESC
LIMIT 1000;
```

iv) Highest reported disease per week (overall)

```
SELECT date_trunc('day', CAST(week_start AS TIMESTAMP)) AS week_start,
       highest_reported_disease AS highest_reported_disease,
       sum(cases) AS "SUM(cases)"
FROM
  (WITH weekly_counts AS
     (SELECT date_trunc('week', visit_date) AS week_start,
             disease,
             COUNT(*) AS cases
      FROM hive.public_health.patient_visits
      GROUP BY 1,
               2),
        ranked AS
     (SELECT week_start,
             disease,
             cases,
             RANK() OVER (PARTITION BY week_start
                          ORDER BY cases DESC) AS rnk
      FROM weekly_counts) SELECT week_start,
                                 disease AS highest_reported_disease,
```

```
                                  cases
   FROM ranked
   WHERE rnk = 1
   ORDER BY week_start DESC) AS virtual_table
GROUP BY date_trunc('day', CAST(week_start AS TIMESTAMP)),
         highest_reported_disease
ORDER BY "SUM(cases)" DESC
LIMIT 100;
```

v) Weekly outbreak detection (all diseases, by county)

```
SELECT county AS county,
       disease AS disease,
       sum(cases_week) AS "SUM(cases_week)"
FROM
  (WITH weekly AS
     (SELECT county,
             disease,
             date_trunc('week', visit_date) AS week_start,
             COUNT(*) AS cases_week
      FROM hive.public_health.patient_visits
      GROUP BY county,
               disease,
               date_trunc('week', visit_date)),
        with_prev AS
     (SELECT county,
             disease,
             week_start,
             cases_week,
             LAG(cases_week) OVER (PARTITION BY county,
                                                disease
                                   ORDER BY week_start) AS prev_week
      FROM weekly) SELECT county,
                          disease,
                          week_start,
                          cases_week,
                          prev_week,
                          ROUND((cases_week - prev_week) * 1.0 /
NULLIF(prev_week, 0), 3) AS pct_change
   FROM with_prev
   WHERE prev_week IS NOT NULL
     AND prev_week >= 20
     AND cases_week >= prev_week * 1.5
   ORDER BY pct_change DESC, cases_week DESC) AS virtual_table
GROUP BY county,
         disease
ORDER BY "SUM(cases_week)" DESC
LIMIT 1000;
```